

ETL para Star Schema - empresa_etl

Flujo: EXTRACT -> TRANSFORM -> LOAD -> Star Schema + KPIs

Que es ETL?

Fase	Descripcion
Extract	Carga los datos crudos desde la fuente
Transform	Limpia y valida los datos
Load	Carga los datos limpios al modelo final

Que es un Star Schema?

Un esquema en estrella organiza los datos en:

- Tablas de Dimensiones (DIM): describen el quien, que, donde
- Tabla de Hechos (FACT): almacena los eventos medibles como ventas

```
dim_cliente
|
dim_empleado -- fact_ventas -- dim_producto
```

Conexion a MySQL

```
In [1]: %load_ext sql

usuario      = 'Daniella'
contrasena   = 'daniella123'
host         = '127.0.0.1'
puerto      = '3306'

%sql mysql+pymysql://Daniella:daniella123@127.0.0.1:3306/
```

Crear la base de datos empresa_etl

```
In [2]: %%sql
DROP DATABASE IF EXISTS empresa_etl;

CREATE DATABASE empresa_etl
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
12 rows affected.
1 rows affected.

Out[2]: []

In [3]: %%sql
USE empresa_etl;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.

Out[3]: []

```

1. EXTRACT - Tablas Staging

En esta fase los datos se depositan en tablas staging sin ninguna transformacion. Estos datos pueden tener NULLs y duplicados, lo cual es normal en datos crudos.

Problema	Tabla
cliente_id = 4 duplicado	stg_clientes
nombre = NULL	stg_clientes, stg_empleados
producto = NULL	stg_productos

```

In [4]: %%sql
DROP TABLE IF EXISTS stg_clientes;
CREATE TABLE stg_clientes (
    cliente_id INT,
    nombre      VARCHAR(100),
    ciudad      VARCHAR(100),
    edad        INT
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.

Out[4]: []

```

```

In [5]: %%sql
INSERT INTO stg_clientes VALUES
(1, 'Carlos', 'San Salvador', 25),
(2, 'Ana',    'Santa Ana',    30),
(3, NULL,    'San Miguel',    22),
(4, 'Pedro', 'La Libertad', 40),
(4, 'Pedro', 'La Libertad', 40);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

Out[5]: []

```

```
In [6]: %%sql
DROP TABLE IF EXISTS stg_productos;
CREATE TABLE stg_productos (
    producto_id INT,
    producto     VARCHAR(100),
    categoria    VARCHAR(100),
    precio       DECIMAL(10,2)
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
```

Out[6]: []

```
In [7]: %%sql
INSERT INTO stg_productos VALUES
(1, 'Laptop', 'Tecnologia', 1200.00),
(2, 'Mouse',  'Tecnologia',  25.00),
(3, 'Silla',  'Muebles',    150.00),
(4, NULL,    'Tecnologia',  75.00);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[7]: []

```
In [8]: %%sql
DROP TABLE IF EXISTS stg_empleados;
CREATE TABLE stg_empleados (
    empleado_id INT,
    nombre       VARCHAR(100),
    departamento VARCHAR(100)
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
```

Out[8]: []

```
In [9]: %%sql
INSERT INTO stg_empleados VALUES
(1, 'Maria', 'Ventas'),
(2, 'Jose',  'Ventas'),
(3, NULL,    'Marketing');

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[9]: []

```
In [10]: %%sql
DROP TABLE IF EXISTS stg_ventas;
CREATE TABLE stg_ventas (
    venta_id     INT,
    cliente_id   INT,
    producto_id  INT,
```

```
    empleado_id INT,  
    cantidad     INT  
);
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/  
0 rows affected.  
0 rows affected.
```

Out[10]: []

```
In [11]: %%sql  
INSERT INTO stg_ventas VALUES  
(1, 1, 1, 1, 2),  
(2, 2, 2, 1, 5),  
(3, 3, 3, 2, 1),  
(4, 4, 1, 2, 1),  
(5, 1, 2, 1, 3);
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/  
5 rows affected.
```

Out[11]: []

Ver datos crudos

```
In [12]: %%sql  
SELECT 'stg_clientes' AS tabla, COUNT(*) AS filas FROM stg_clientes  
UNION ALL  
SELECT 'stg_productos', COUNT(*) FROM stg_productos  
UNION ALL  
SELECT 'stg_empleados', COUNT(*) FROM stg_empleados  
UNION ALL  
SELECT 'stg_ventas',    COUNT(*) FROM stg_ventas;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/  
4 rows affected.
```

Out[12]:

tabla	filas
-------	-------

stg_clientes	5
stg_productos	4
stg_empleados	3
stg_ventas	5

```
In [13]: %%sql  
SELECT * FROM stg_clientes;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/  
5 rows affected.
```

Out[13]: **cliente_id nombre ciudad edad**

1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	None	San Miguel	22
4	Pedro	La Libertad	40
4	Pedro	La Libertad	40

In [14]: **%%sql**
SELECT * FROM stg_productos;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

Out[14]: **producto_id producto categoria precio**

1	Laptop	Tecnologia	1200.00
2	Mouse	Tecnologia	25.00
3	Silla	Muebles	150.00
4	None	Tecnologia	75.00

In [15]: **%%sql**
SELECT * FROM stg_empleados;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.

Out[15]: **empleado_id nombre departamento**

1	Maria	Ventas
2	Jose	Ventas
3	None	Marketing

In [16]: **%%sql**
SELECT * FROM stg_ventas;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

Out[16]: **venta_id cliente_id producto_id empleado_id cantidad**

1	1	1	1	2
2	2	2	1	5
3	3	3	2	1
4	4	1	2	1
5	1	2	1	3

2. TRANSFORM - Limpieza de datos

Se aplican reglas para corregir los datos antes de cargarlos al modelo final.

Problema	Solucion SQL
Nombres NULL en clientes/empleados	COALESCE(nombre, 'Sin Nombre')
Nombre de producto NULL	COALESCE(producto, 'Producto Desconocido')
Filas duplicadas	SELECT DISTINCT
Campo total faltante	cantidad x precio via JOIN

```
In [17]: %%sql
DROP TABLE IF EXISTS clientes_clean;
CREATE TABLE clientes_clean AS
SELECT DISTINCT
    cliente_id,
    COALESCE(nombre, 'Sin Nombre') AS nombre,
    ciudad,
    edad
FROM stg_clientes;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.
```

Out[17]: []

```
In [18]: %%sql
DROP TABLE IF EXISTS productos_clean;
CREATE TABLE productos_clean AS
SELECT DISTINCT
    producto_id,
    COALESCE(producto, 'Producto Desconocido') AS producto,
    categoria,
    precio
FROM stg_productos;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.
```

Out[18]: []

```
In [19]: %%sql
DROP TABLE IF EXISTS empleados_clean;
CREATE TABLE empleados_clean AS
SELECT DISTINCT
    empleado_id,
    COALESCE(nombre, 'Sin Nombre') AS nombre,
```

```
departamento
FROM stg_empleados;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
3 rows affected.
```

Out[19]: []

```
In [20]: %%sql
DROP TABLE IF EXISTS ventas_transformadas;
CREATE TABLE ventas_transformadas AS
SELECT
    v.venta_id,
    v.cliente_id,
    v.producto_id,
    v.empleado_id,
    v.cantidad,
    p.precio,
    (v.cantidad * p.precio) AS total
FROM stg_ventas v
JOIN productos_clean p ON v.producto_id = p.producto_id;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
5 rows affected.
```

Out[20]: []

Verificar resultados de la limpieza

```
In [21]: %%sql
-- Antes: 5 filas con duplicado y NULL | Despues: 4 filas limpias
SELECT * FROM clientes_clean;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[21]:

cliente_id	nombre	ciudad	edad
------------	--------	--------	------

1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	Sin Nombre	San Miguel	22
4	Pedro	La Libertad	40

```
In [22]: %%sql
SELECT * FROM productos_clean;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[22]:

	producto_id	producto	categoria	precio
	1	Laptop	Tecnologia	1200.00
	2	Mouse	Tecnologia	25.00
	3	Silla	Muebles	150.00
	4	Producto Desconocido	Tecnologia	75.00

In [23]:

```
%%sql
SELECT * FROM empleados_clean;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.

Out[23]:

	empleado_id	nombre	departamento
	1	Maria	Ventas
	2	Jose	Ventas
	3	Sin Nombre	Marketing

In [24]:

```
%%sql
SELECT * FROM ventas_transformadas;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

Out[24]:

	venta_id	cliente_id	producto_id	empleado_id	cantidad	precio	total
	1	1	1	1	2	1200.00	2400.00
	2	2	2	1	5	25.00	125.00
	3	3	3	2	1	150.00	150.00
	4	4	1	2	1	1200.00	1200.00
	5	1	2	1	3	25.00	75.00

3. LOAD - Dimensiones

Las tablas de dimension son el contexto descriptivo del Star Schema. Cada una tiene una clave primaria (PK) que la vincula a la tabla de hechos.

dim_cliente

In [25]:

```
%%sql
DROP TABLE IF EXISTS dim_cliente;
CREATE TABLE dim_cliente (
    cliente_id INT PRIMARY KEY,
```



```

    nombre    VARCHAR(100),
    ciudad    VARCHAR(100),
    edad      INT
);

INSERT INTO dim_cliente
SELECT * FROM clientes_clean;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
4 rows affected.

```

Out[25]: []

```

In [26]: %%sql
SELECT * FROM dim_cliente;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

```

Out[26]:

cliente_id	nombre	ciudad	edad
1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	Sin Nombre	San Miguel	22
4	Pedro	La Libertad	40

dim_producto

```

In [27]: %%sql
DROP TABLE IF EXISTS dim_producto;
CREATE TABLE dim_producto (
    producto_id INT PRIMARY KEY,
    producto    VARCHAR(100),
    categoria    VARCHAR(100),
    precio       DECIMAL(10,2)
);

INSERT INTO dim_producto
SELECT * FROM productos_clean;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
4 rows affected.

```

Out[27]: []

```

In [28]: %%sql
SELECT * FROM dim_producto;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

```

Out[28]:

	producto_id	producto	categoria	precio
	1	Laptop	Tecnologia	1200.00
	2	Mouse	Tecnologia	25.00
	3	Silla	Muebles	150.00
	4	Producto Desconocido	Tecnologia	75.00

dim_empleado

In [29]:

```
%%sql
DROP TABLE IF EXISTS dim_empleado;
CREATE TABLE dim_empleado (
    empleado_id INT PRIMARY KEY,
    nombre      VARCHAR(100),
    departamento VARCHAR(100)
);

INSERT INTO dim_empleado
SELECT * FROM empleados_clean;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
3 rows affected.

Out[29]: []

In [30]:

```
%%sql
SELECT * FROM dim_empleado;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.

Out[30]:

	empleado_id	nombre	departamento
	1	Maria	Ventas
	2	Jose	Ventas
	3	Sin Nombre	Marketing

4. LOAD - Fact Table

La tabla de hechos es el centro del Star Schema. Contiene las metricas medibles (cantidad, precio, total) y las claves foraneas (FK) que apuntan a cada dimension.

In [31]:

```
%%sql
DROP TABLE IF EXISTS fact_ventas;
CREATE TABLE fact_ventas (
    venta_id INT PRIMARY KEY,
```

```

    cliente_id INT,
    producto_id INT,
    empleado_id INT,
    cantidad INT,
    precio DECIMAL(10,2),
    total DECIMAL(10,2),

    FOREIGN KEY (cliente_id) REFERENCES dim_cliente(cliente_id),
    FOREIGN KEY (producto_id) REFERENCES dim_producto(producto_id),
    FOREIGN KEY (empleado_id) REFERENCES dim_empleado(empleado_id)
);

INSERT INTO fact_ventas
SELECT venta_id, cliente_id, producto_id, empleado_id, cantidad, precio, total
FROM ventas_transformadas;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
5 rows affected.

```

Out[31]: []

```

In [32]: %%sql
SELECT * FROM fact_ventas;

```

```

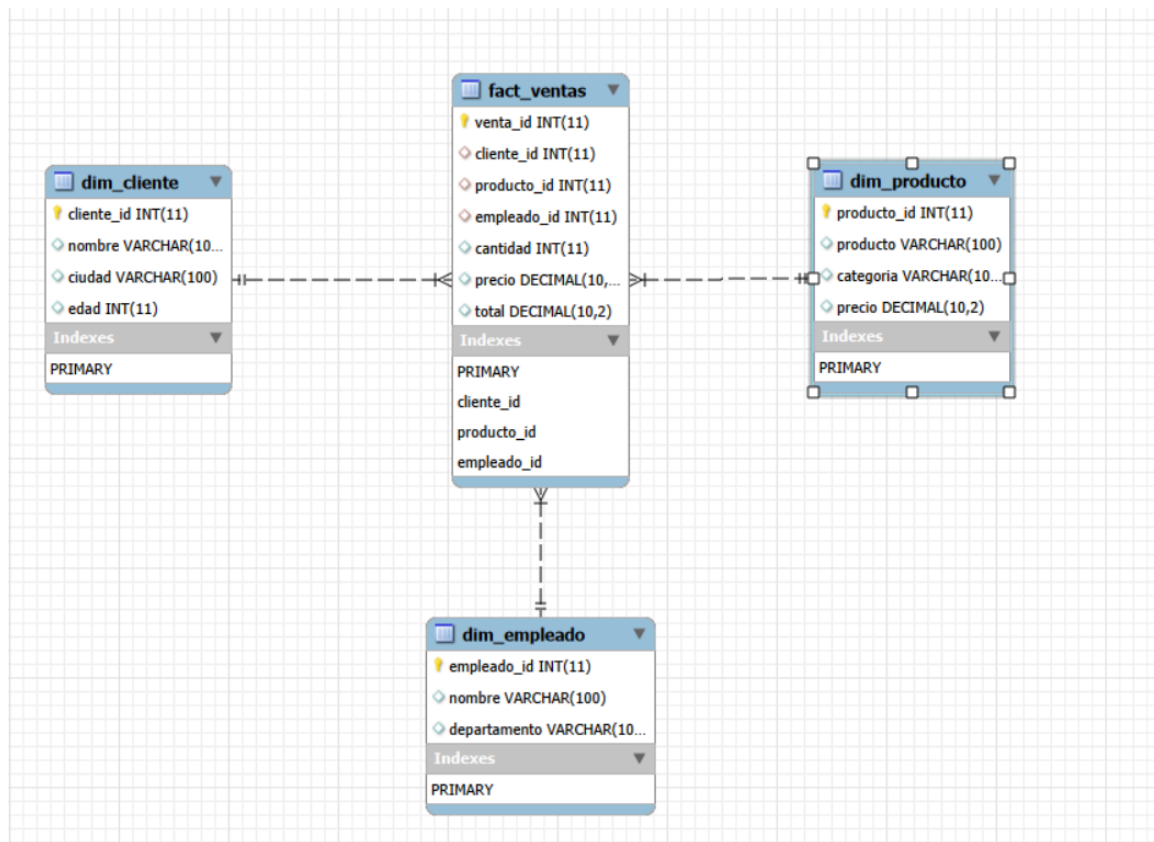
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

```

Out[32]:

venta_id	cliente_id	producto_id	empleado_id	cantidad	precio	total
1	1	1	1	2	1200.00	2400.00
2	2	2	1	5	25.00	125.00
3	3	3	2	1	150.00	150.00
4	4	1	2	1	1200.00	1200.00
5	1	2	1	3	25.00	75.00

5. Diagrama del Star Schema



En el centro — fact_ventas: esta es la tabla más importante. Guarda cada venta que ocurrió, cuánto se vendió, el precio y el total. No repite información de clientes ni productos, solo guarda los IDs para conectarse con las otras tablas.

En los brazos — las dimensiones: son dim_cliente, dim_producto y dim_empleado. Cada una describe el contexto de la venta, quién compró, qué se vendió y quién lo vendió. Todo el detalle vive directamente dentro de cada tabla: la ciudad está en dim_cliente, la categoría está en dim_producto, el departamento está en dim_empleado. y se conectan a través de las Foreign Keys (FK). La fact_ventas tiene cliente_id, producto_id y empleado_id, y con esos IDs puede ir a buscar el detalle a cada dimensión.

6. KPIs - Consultas analíticas

KPI 1 - Ingreso total y promedio por venta

```
In [33]: %%sql
SELECT
    COUNT(*)      AS num_ventas,
    SUM(total)    AS ingreso_total,
    AVG(total)    AS promedio_venta
FROM fact_ventas;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.
```

Out[33]: **num_ventas ingreso_total promedio_venta**

5	3950.00	790.000000
---	---------	------------

KPI 2 - Ventas por ciudad

```
In [34]: %%sql
SELECT
    c.ciudad,
    SUM(f.total) AS ventas
FROM fact_ventas f
JOIN dim_cliente c ON f.cliente_id = c.cliente_id
GROUP BY c.ciudad
ORDER BY ventas DESC;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[34]: **ciudad ventas**

San Salvador	2475.00
La Libertad	1200.00
San Miguel	150.00
Santa Ana	125.00

KPI 3 - Top clientes por monto comprado

```
In [35]: %%sql
SELECT
    c.nombre AS cliente,
    SUM(f.total) AS total_comprado
FROM fact_ventas f
JOIN dim_cliente c ON f.cliente_id = c.cliente_id
GROUP BY c.nombre
ORDER BY total_comprado DESC;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[35]: **cliente total_comprado**

Carlos	2475.00
Pedro	1200.00
Sin Nombre	150.00
Ana	125.00

KPI 4 - Ingresos por categoria de producto

```
In [36]: %%sql
SELECT
    p.categoria,
    SUM(f.total) AS ingresos
FROM fact_ventas f
JOIN dim_producto p ON f.producto_id = p.producto_id
GROUP BY p.categoria
ORDER BY ingresos DESC;
```

```
* mysql+pymysql://Daniella:**@127.0.0.1:3306/
2 rows affected.
```

```
Out[36]: categoria  ingresos
Tecnologia    3800.00
Muebles       150.00
```

KPI 5 - Rendimiento de empleados

```
In [37]: %%sql
SELECT
    e.nombre      AS empleado,
    SUM(f.total)  AS ventas_generadas
FROM fact_ventas f
JOIN dim_empleado e ON f.empleado_id = e.empleado_id
GROUP BY e.nombre
ORDER BY ventas_generadas DESC;
```

```
* mysql+pymysql://Daniella:**@127.0.0.1:3306/
2 rows affected.
```

```
Out[37]: empleado  ventas_generadas
Maria          2600.00
Jose           1350.00
```

7. Resumen del proceso ETL

Paso	Fase	Accion
1	Extract	Datos crudos en tablas staging en MySQL
2	Transform	COALESCE para NULLs, DISTINCT para duplicados, JOIN para calcular total
3	Load DIM	dim_cliente, dim_producto, dim_empleado con claves primarias
4	Load FACT	fact_ventas con FK y metricas de negocio

Paso	Fase	Accion
5	Star Schema	Diagrama visual del modelo dimensional
6	KPIs	5 consultas analiticas con graficos

Problemas detectados y resueltos

Problema	Tabla	Solucion
Fila duplicada Pedro id=4	stg_clientes	SELECT DISTINCT
nombre = NULL	stg_clientes, stg_empleados	COALESCE(nombre, 'Sin Nombre')
producto = NULL	stg_productos	COALESCE(producto, 'Producto Desconocido')
Campo total no existia	stg_ventas	cantidad x precio via JOIN